



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE, INDIA

Unit 3

BASIC PROCESSING MODULE

MISSION

CHRIST is a nurturing ground for an individual's holistic development to make effective contribution to

VISION

Excellence and Service

CORE VALUES

Faith in God | Moral Uprightness
Love of Fellow Beings

UNIT 3 -SYLLABUS

BASIC PROCESSING MODULE: Fundamental concepts – Execution of a complete instruction – Multiple bus organization – Hardwired control – Micro programmed control -Basic concepts – Data hazards – Instruction hazards – Influence on Instruction sets – Data path and control consideration – Superscalar operation.

CPU- CENTRAL PROCESSING UNIT

FUNCTIONS OF CPU:

CPU carries out all forms of data processing tasks.

It saves information, intermediate results and instructions.

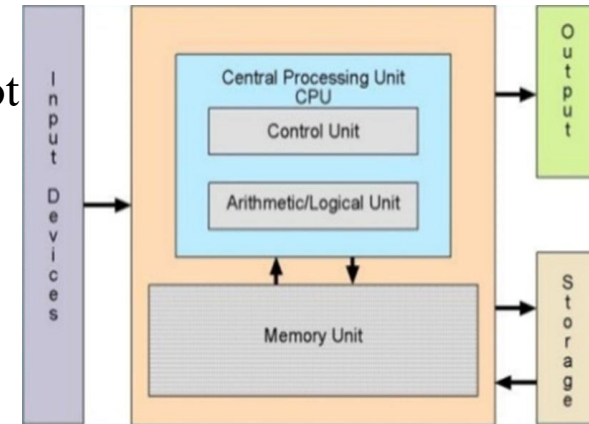
CPU monitors the functionality of all computer components.

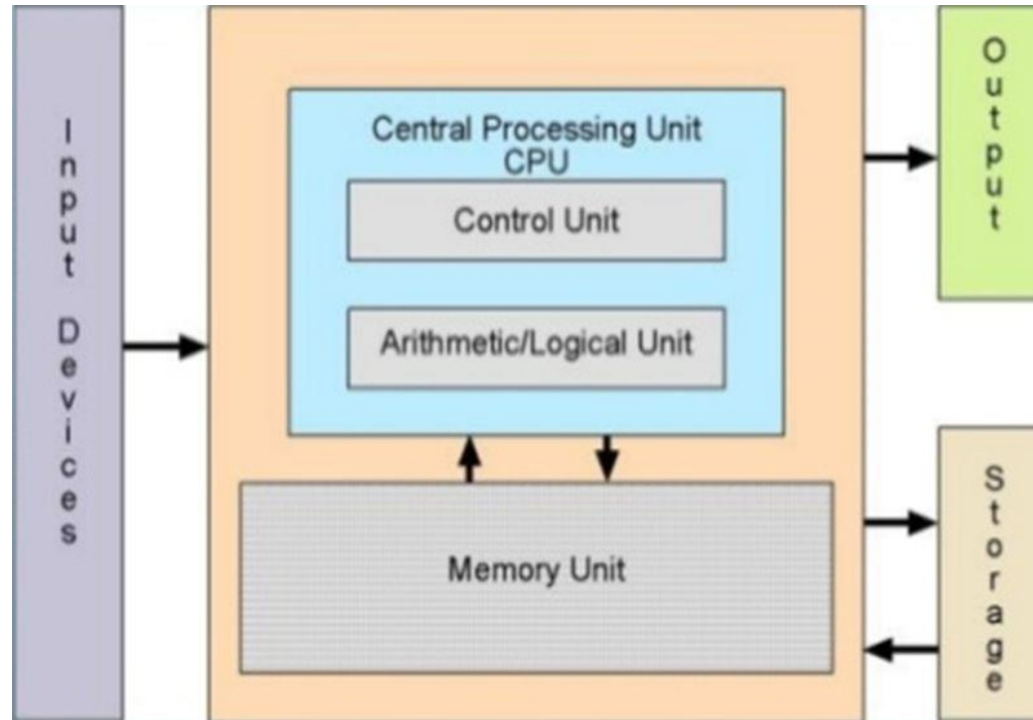
COMPONENTS OF CPU:

Register: Stores data , result and speeds up the operation

Control unit: This unit monitors all computing processes but does not execute actual data processing.

Arithmetic Logic Unit (ALU): This does all the calculations and makes the decisions.





FUNDAMENTAL CONCEPTS OF BASIC PROCESSING UNIT.

Processor fetches one instruction at a time and perform the specified operation.

Instructions are fetched from successive memory location until a branch/jump instruction is encountered.

The address of the next instruction to be executed is tracked by the **Program Counter (PC)** register.

Instruction Register (IR) contains instruction that is currently executed.

Instruction execution happens in three phases:

Fetch: Fetch the instruction from the specified memory

Decode: Determined the opcode and the operands

Execute: Run the instruction

FUNDAMENTAL CONCEPTS-single bus organization.

Fetch the contents of memory location pointed by the PC. The contents of this memory location is loaded to the IR-**Fetch phase**

$$IR \leftarrow [[PC]]$$

Increment the PC by 4 (assume the word size as 4)

$$PC = [PC] + 4$$

Carry out the actions specified by the instruction in the IR-**Execution phase**

MDR: Two inputs and two outputs since data can be loaded from memory or processor bus.

MAR: Input line is connected to internal bus and output line to external bus.

Control lines: connected to instruction decoder and control logic block to issue control signals.

R0-R(n-1): General Purpose registers whose numbers vary between processors.

TEMP, Y and Z: temporary registers used by the processor during instruction execution.

The registers, the ALU, and the interconnecting bus are collectively referred to as the **Datapath**.

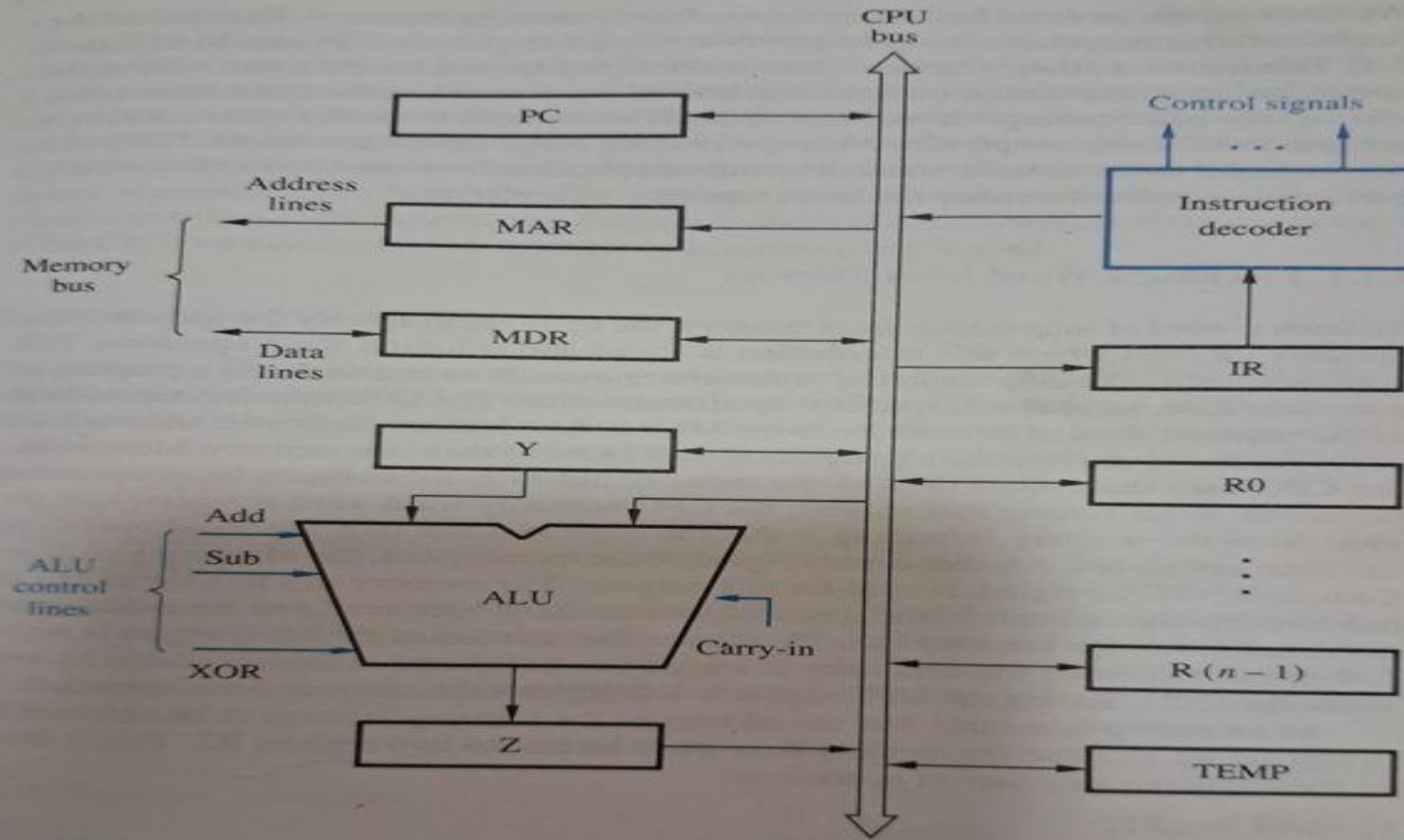


FIGURE 3.1
Single-bus organization of the data paths inside the CPU

Execution of a Complete Instruction

Consider the instruction
Add (R3), R1

Executing this instruction requires the following actions:

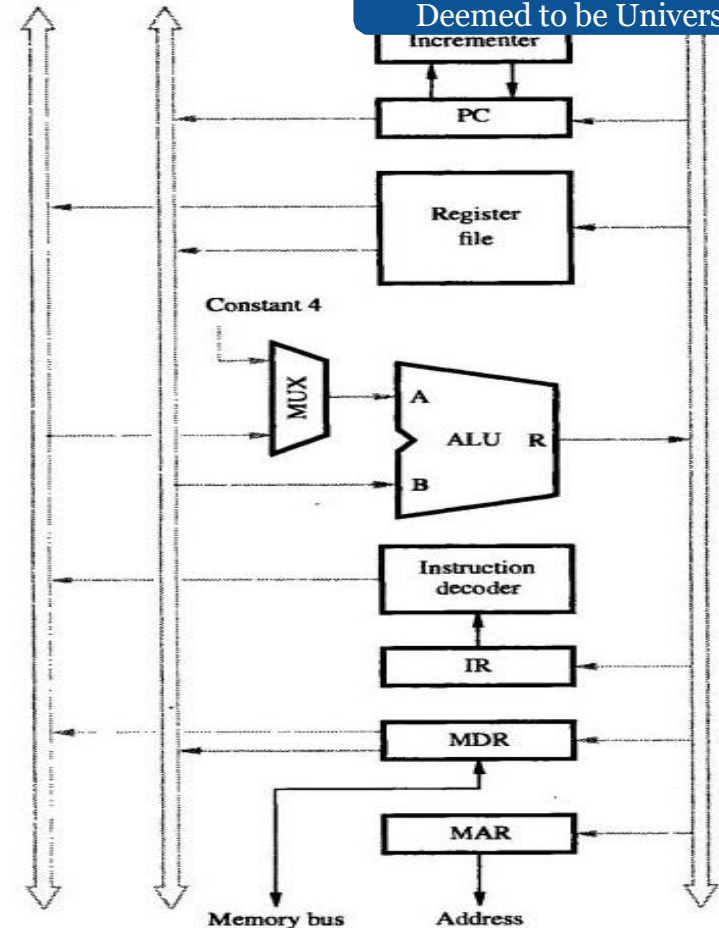
- Fetch the instruction
- Fetch the first operand
(the contents of the memory location pointed to by R3)
Perform the addition
- Load the result into R1

Multiple-Bus Organization

Till now, we have considered the simple single-bus structure of processing unit to illustrate the basic ideas.

The resulting control sequence to execute an instruction is quite long because only one data item can be transferred over the bus in a clock cycle.

To reduce the number of steps needed, most commercial processors provide multiple internal paths that enable several transfers to take place in parallel.



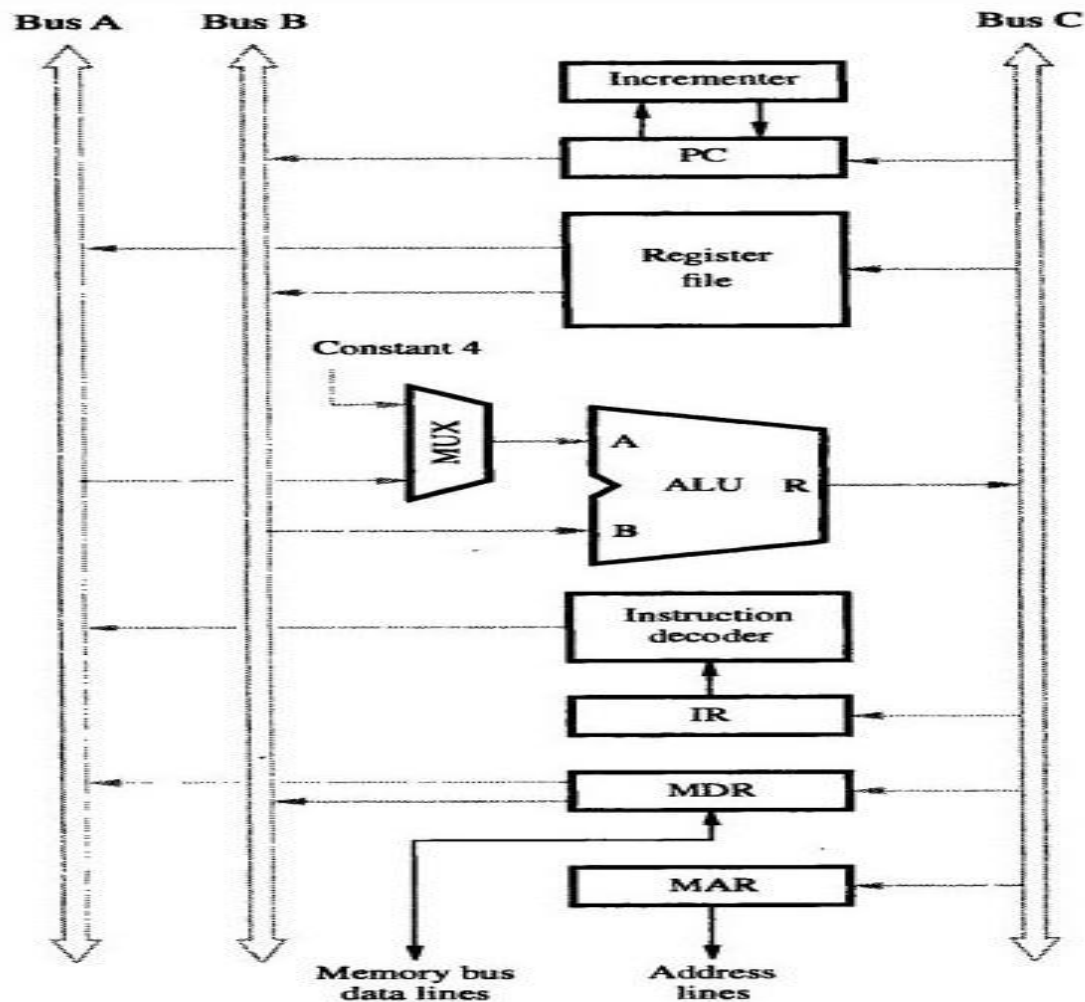


Figure 7.8 Three-bus organization of the datapath.

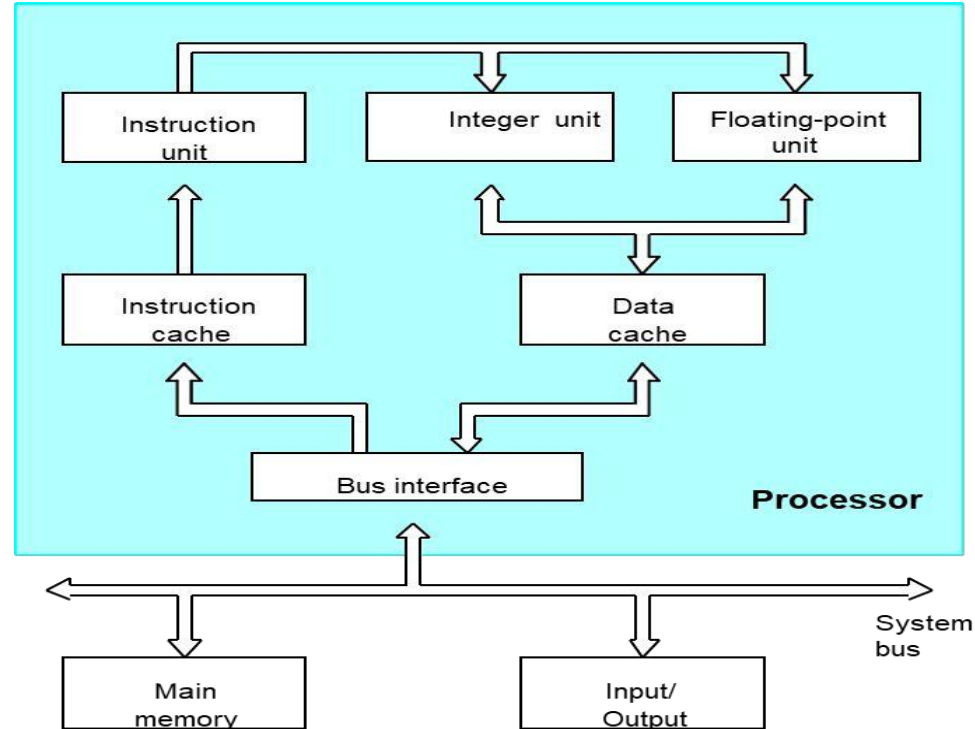
Multiple-Bus Organization

- ❑ All general-purpose registers are combined into a single block called the **register file**.
- ❑ The register file is said to have three ports.
- ❑ There are two outputs, allowing the contents of two different registers to be accessed simultaneously and have their contents placed on buses A and B.
- ❑ The third port allows the data on bus C to be loaded into a third register during the same clock cycle.
- ❑ Buses A and B are used to transfer the source operands to the A and B inputs of the ALU, where an arithmetic or logic operation may be performed.
- ❑ The result is transferred to the destination over bus C.

Multiple-Bus Organization

- ❑ If needed, the ALU may simply pass one of its two input operands unmodified to bus C.
- ❑ We will call the ALU control signals for such an operation $R=A$ or $R=B$.
- ❑ The three-bus arrangement obviates the need for registers Y and Z as required in single-bus structure processing unit.
- ❑ A second feature in Multiple-Bus Organization is the introduction of the Incrementor unit, which is used to increment the PC by 4.
- ❑ Using the Incrementer eliminates the need to add 4 to the PC using the main ALU.
- ❑ The source for the constant 4 at the ALU input multiplexer is still useful.

A Complete Processor



Block diagram of a complete processor

A Complete Processor

This structure has an instruction unit that fetches instructions from an instruction cache or from the main memory when the desired instructions are not already in the cache.

It has separate processing units to deal with integer data and floating point data .

A data cache is inserted between these units and the main memory.

Using separate caches for instructions and data is common practice in many processors today. Other processors use a single cache that stores both instructions and data.

The processor is connected to the system bus and, hence, to the rest of the computer, by means of a bus interface.

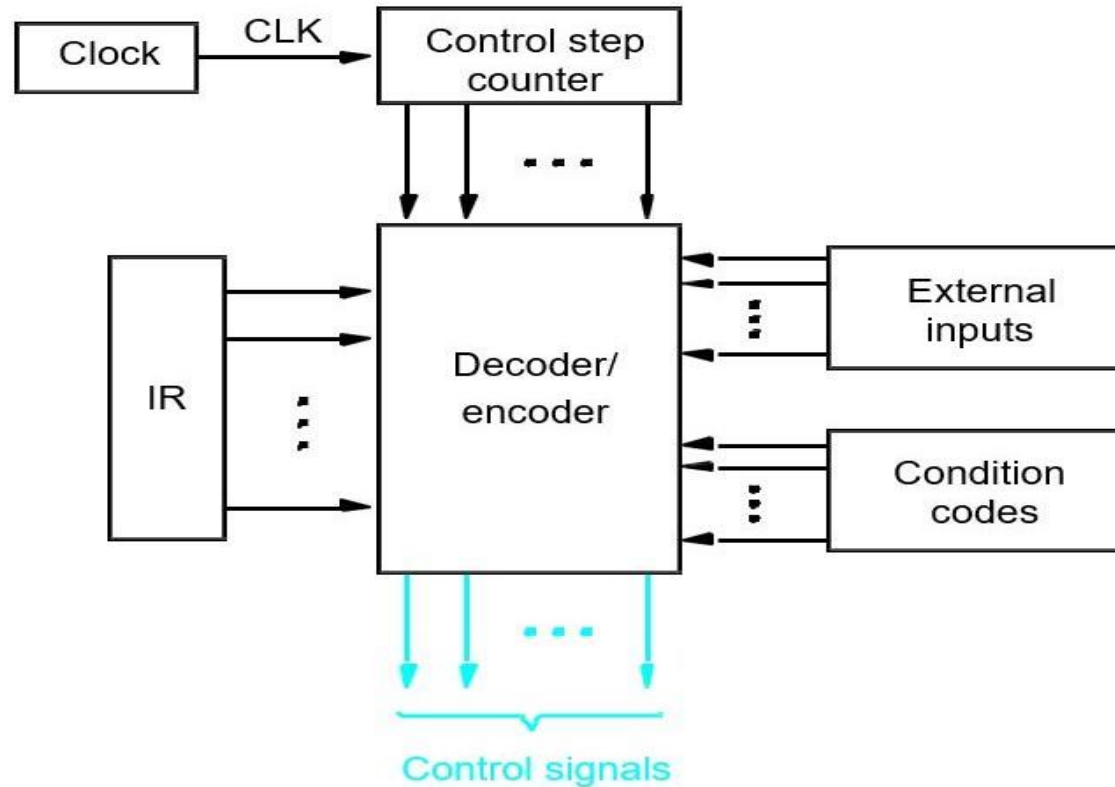
Hardwired Control

To execute instructions, the processor must have some means of generating the control signals needed in the proper sequence.

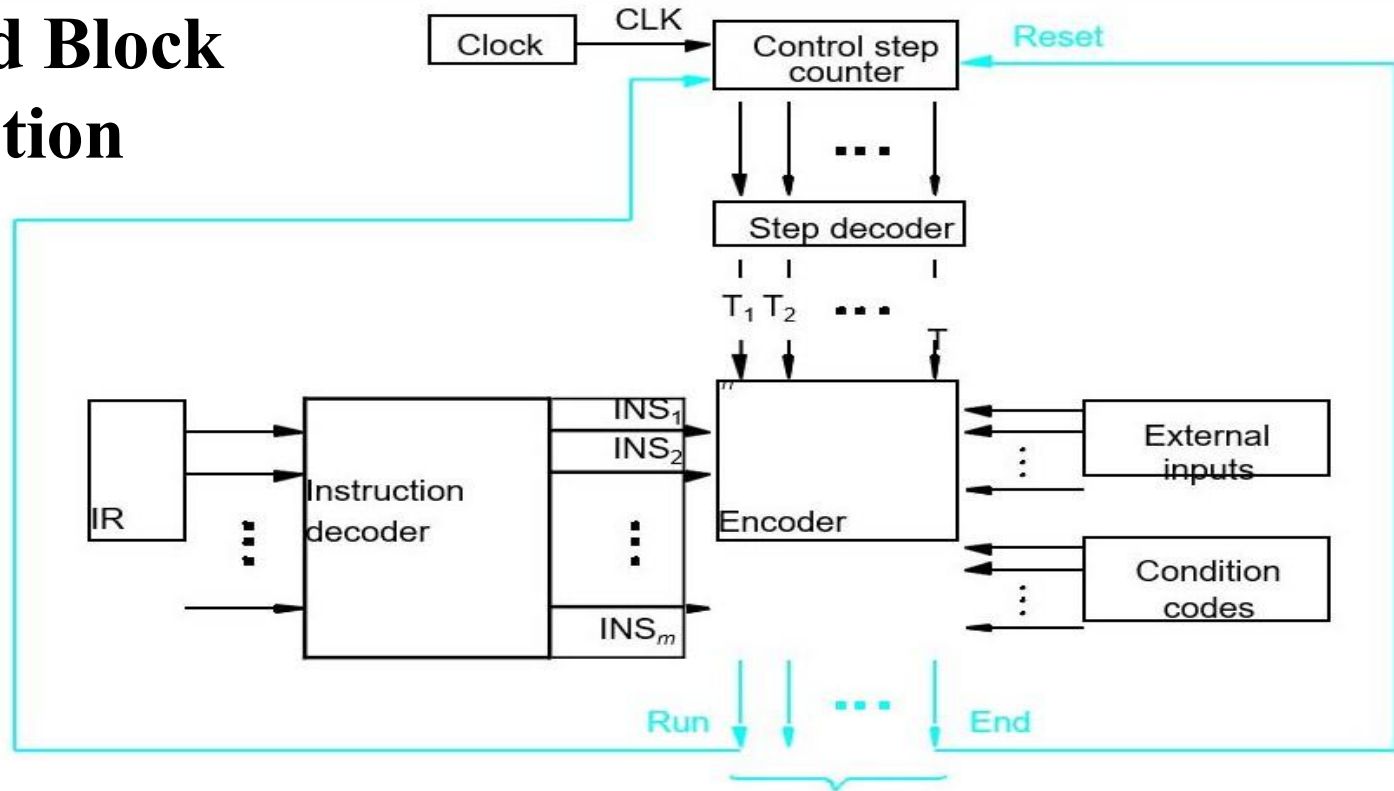
Two categories:
hardwired control and
microprogrammed control

Hardwired system can operate at high speed; but with little flexibility.

Control Unit Organization



Detailed Block Description



- The step decoder provides a separate signal line for each step, or time slot, in the control sequence.
- Similarly, the output of the instruction decoder consists of a separate line for each machine instruction.
- For any instruction loaded in the IR, one of the output lines INS_1 through INS_m is set to 1, and all other lines are set to 0.
- The input signals to the encoder block are combined to generate the individual control signals Yin, PCout, Add, End, and so on

Microprogrammed Control

A control unit whose binary control variables are stored in memory is called a micro programmed control unit.

Control signals

Group of bits used to select paths in multiplexers, decoders, arithmetic logic units

Control variables

Binary variables specify microoperations

Certain microoperations initiated while others idle

Control word

is a word whose individual bits represent the various control signals

Control memory

The micro routines for all instructions in the instruction set of a computer are stored in a special memory called the control store/control memory

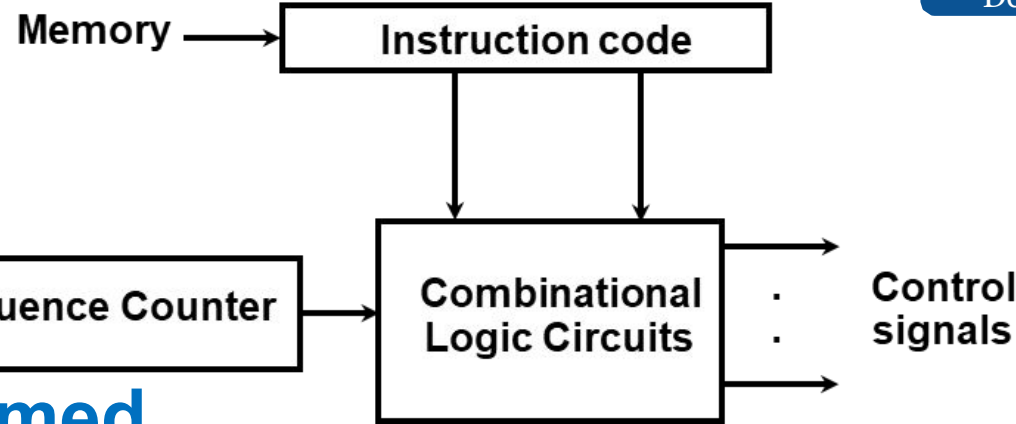
Microinstructions

A sequence of CWs corresponding to the control sequence of a machine instruction constitutes the micro routine for that instruction, and the individual control words in this micro routine are referred to as microinstructions.

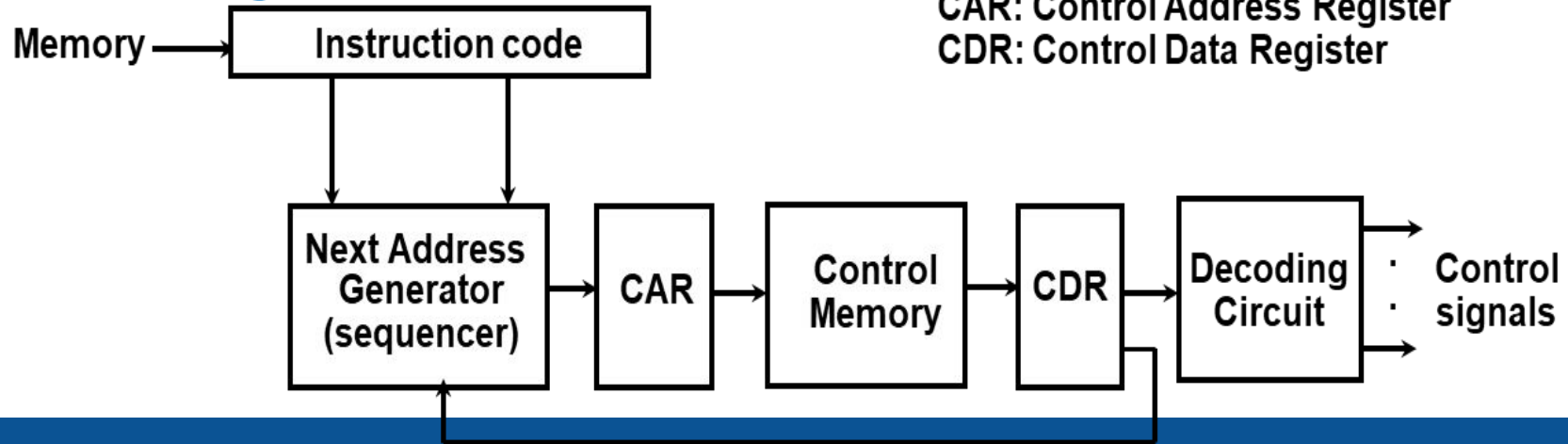
Microprogram

Sequence of microinstructions

Hardwired



Microprogrammed



CAR: Control Address Register
CDR: Control Data Register

Control Memory

Read-only memory (ROM)

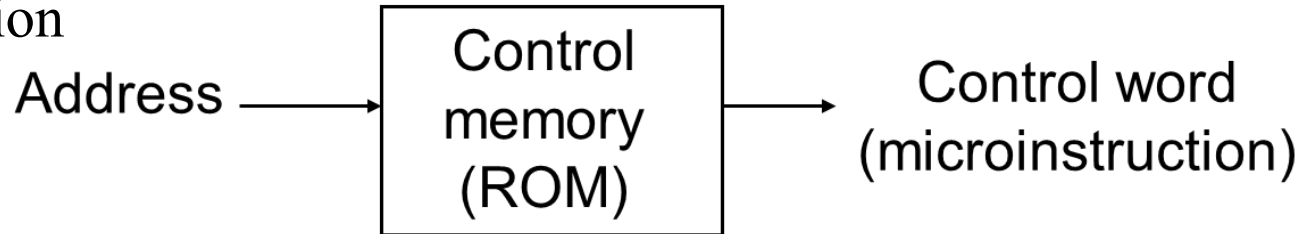
Content of word in ROM at given address specifies microinstruction

Each computer instruction initiates series of microinstructions (microprogram) in control memory.

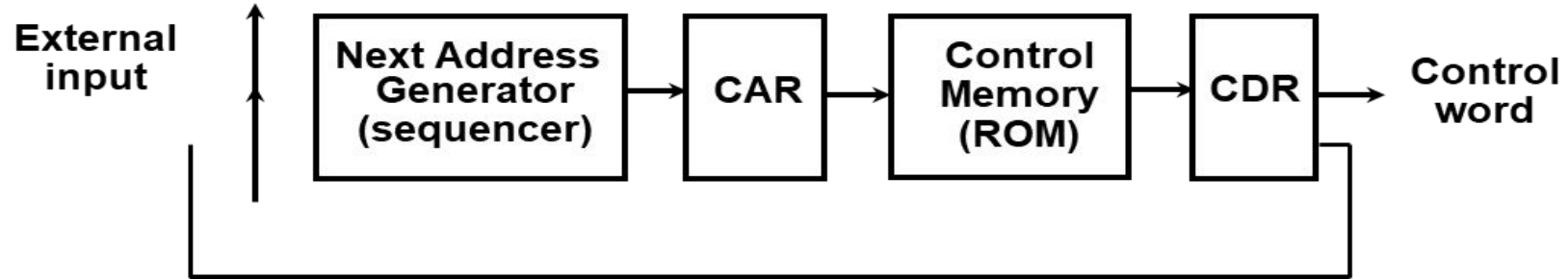
These microinstructions generate microoperations to Fetch instruction from main memory

Evaluate effective address

Execute operation specified by instruction Return control to fetch phase for next instruction



Microprogrammed Control Organization



Control memory

Contains microprograms (set of microinstructions)

Microinstruction contains

Bits initiate microoperations

Bits determine address of next microinstruction

Control address register (CAR)

Specifies address of next microinstruction

Next address generator (microprogram sequencer)

Determines address sequence for control memory.

Microprogram sequencer functions

Increment CAR by one.

Transfer external address into CAR

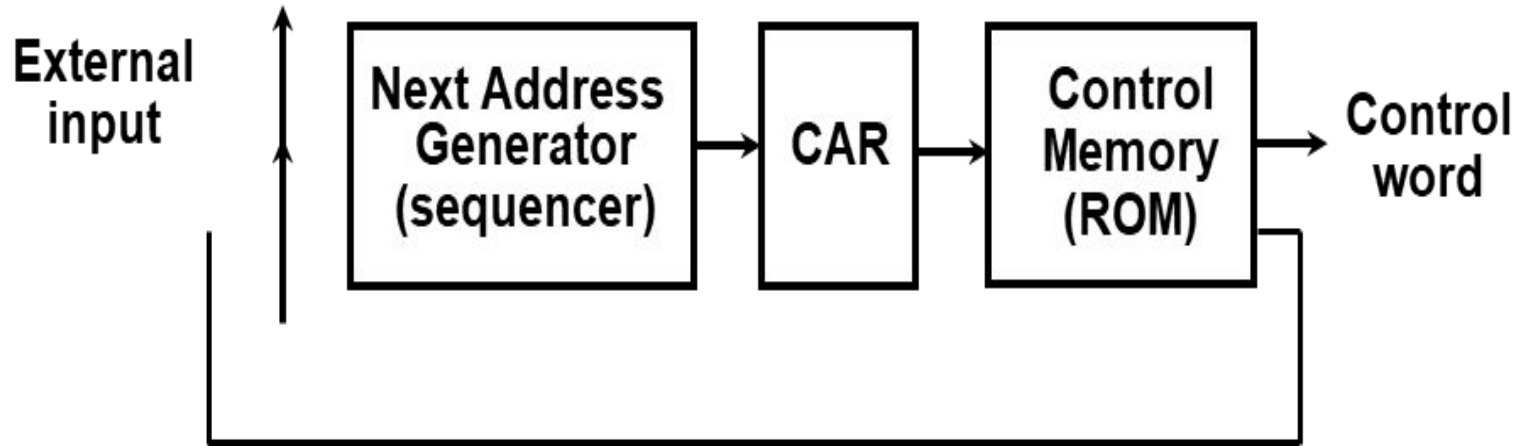
Load initial address into CAR to start control operations

Control data register (CDR)- or pipeline register

Holds microinstruction read from control memory

Allows execution of microoperations specified by control word simultaneously with generation of next microinstruction.

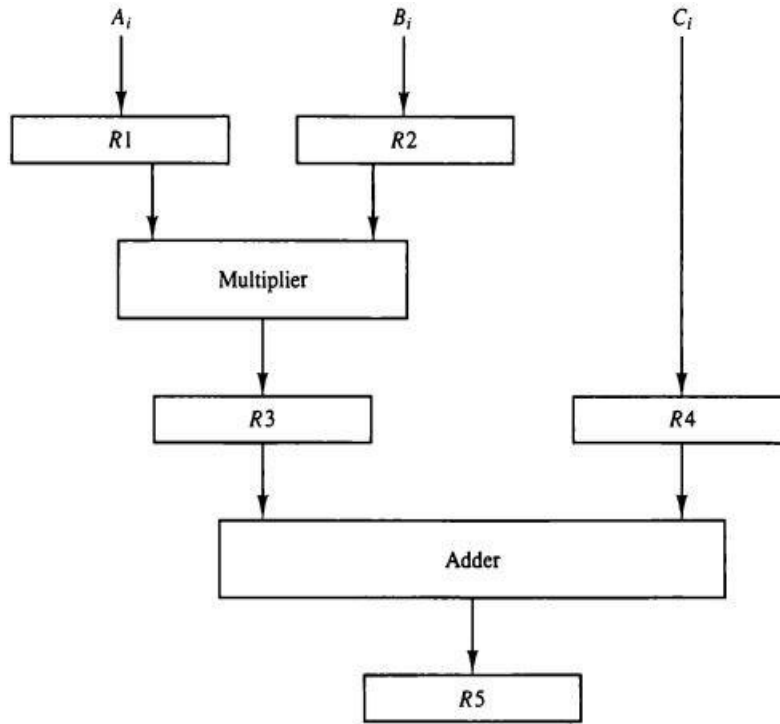
Control unit can operate without CDR



Pipelining

- Pipelining creates and organizes a pipeline of instructions the processor can execute in parallel.
- **Pipelining** is a process of arrangement of hardware elements of the CPU such that its overall performance is increased.
- Execution of more than one instruction takes place in a pipelined processor.
- Types
 1. Arithmetic Pipeline
 2. Instruction Pipeline

Arithmetic pipeline Example--1



$$A_i * B_i + C_i \quad \text{for } i = 1, 2, 3, \dots, 7$$

$R1 \leftarrow A_i, \quad R2 \leftarrow B_i$

Input A_i and B_i

$R3 \leftarrow R1 * R2, \quad R4 \leftarrow C_i$

Multiply and input C_i

$R5 \leftarrow R3 + R4$

Add C_i to product

Arithmetic pipelines are usually found in most of the computers. They are used for floating point operations, multiplication of fixed point numbers

Arithmetic pipeline Example--1

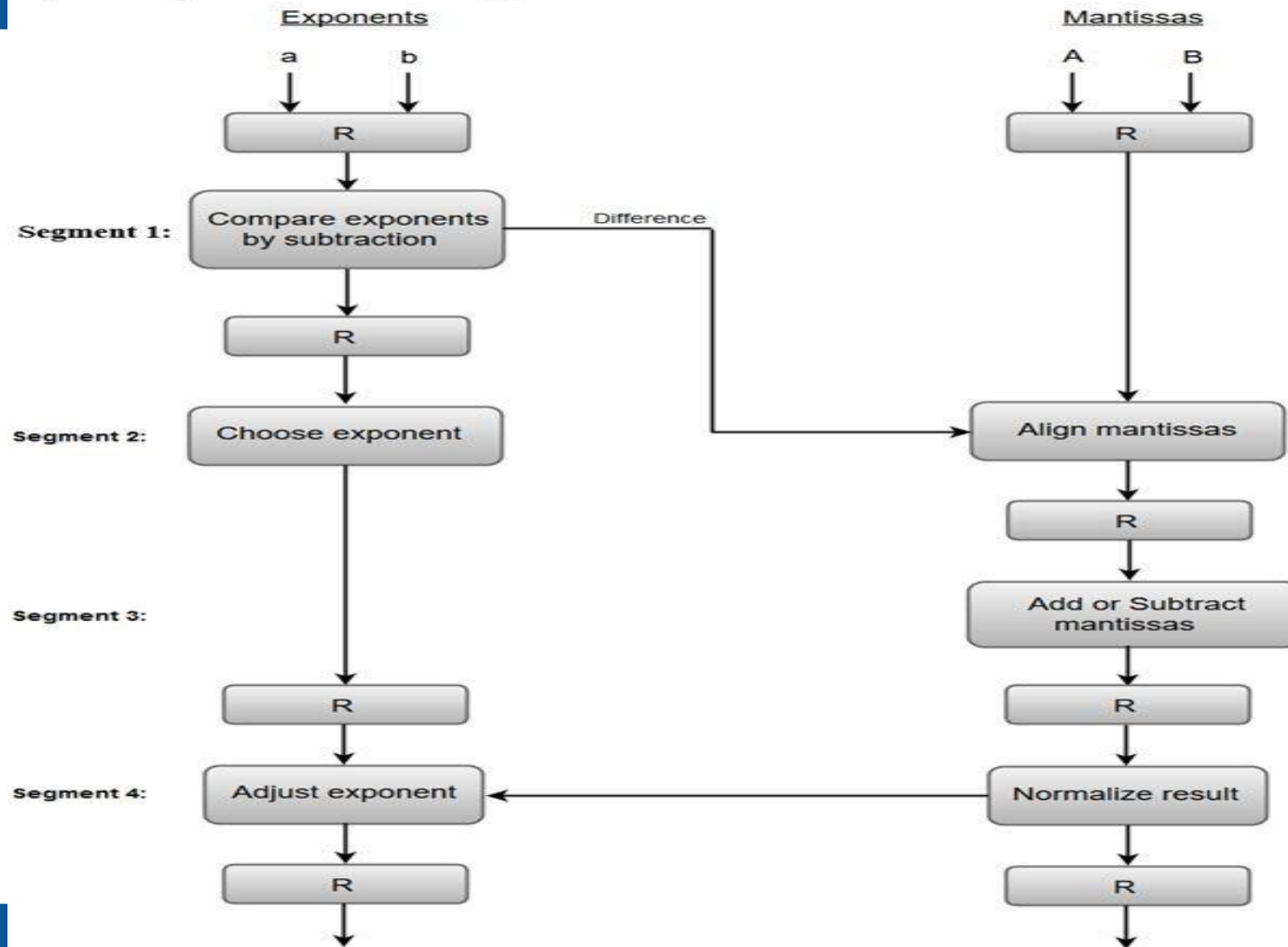
Clock Pulse Number	Segment 1		Segment 2		Segment 3
	<i>R1</i>	<i>R2</i>	<i>R3</i>	<i>R4</i>	<i>R5</i>
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

Arithmetic Pipeline--Example--2

$$X = A \times 2^a$$

$$Y = B \times 2^b$$

1. Compare the exponents.
2. Align the mantissas.
3. Add or subtract the mantissas.
4. Normalize the result.



K--stages in pipeline

Task1	$k \cdot tp$
N-1 tasks	$(n-1)tp$
To complete n tasks	$k+(n-1)tp$

Reasons:::

1. Different segments will take different times to complete their operation
2. Other segments will have a longer wait time.

Non-pipeline= ntn

$$s = ntn / (k + n - 1)tp$$

example

$$t=20 \text{ ns}$$

$$k=4$$

$$n=100$$

$$tp=(4+99)*20=2060 \text{ ns}$$

Total time required to execute n instructions with pipeline = $[k + (n-1)] T$

$$t=20 \text{ ns}$$

$$k=3$$

$$n=5$$

$$tp=(3+4)*20=140 \text{ ns}$$

Instruction cycle

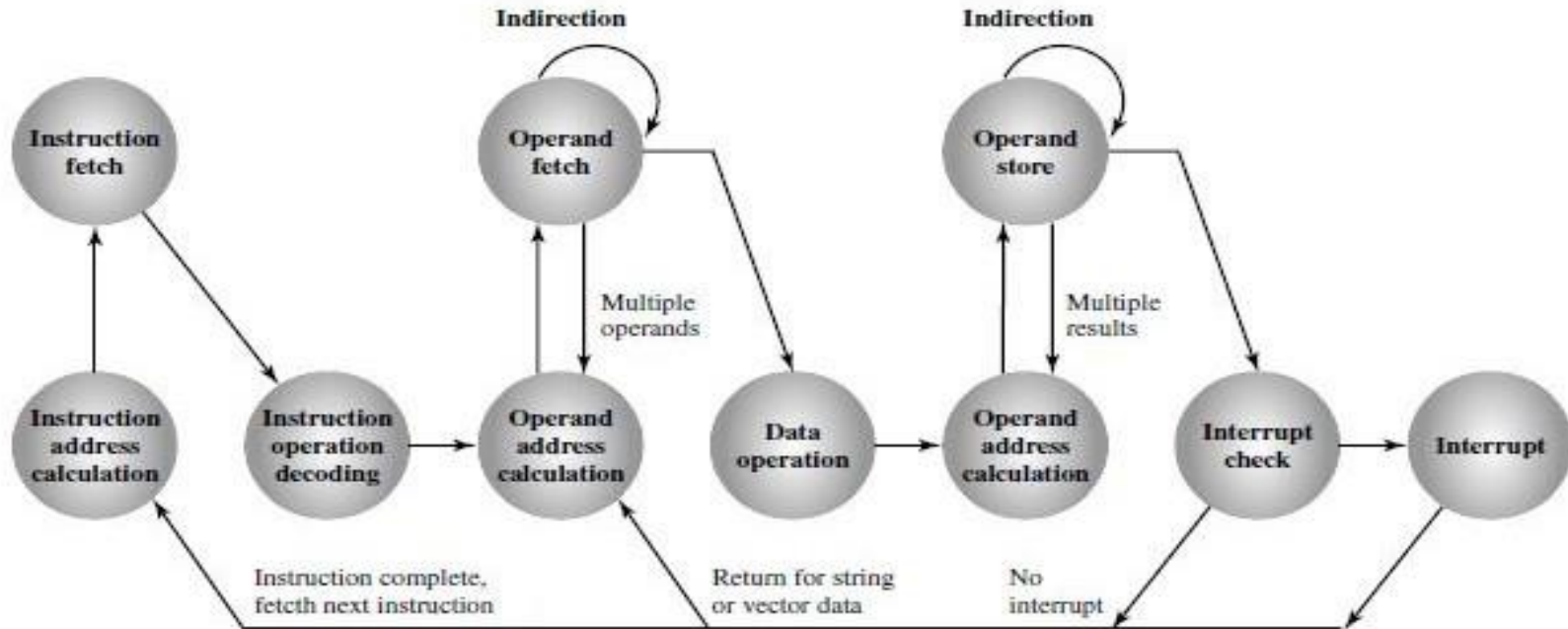


Figure 12.5 Instruction Cycle State Diagram

Instruction pipelining

- Is a method in which the speed of operations are increased beyond sequential execution.
- Here in more than one task is performed at a time.
- One method of gaining speedup with modest hardware cost is the technique of pipelining.
- A task is broken down into multiple steps, and independent processing units are assigned to each step.
- Once a task has completed its initial step, another task may enter that step while the original task moves on to the following step.

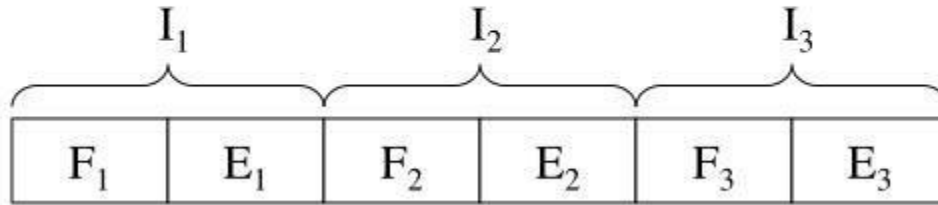
Instruction pipelining

- In a pipeline, new inputs are accepted at one end before previously accepted inputs appear as outputs at the other end.

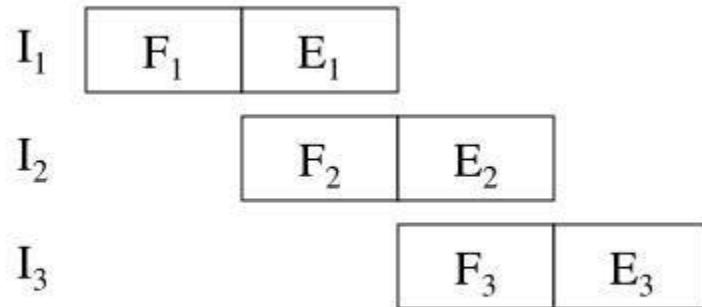
Processors that use instruction pipeline

- 80x86
- Pentium
- ARM

Pipelining

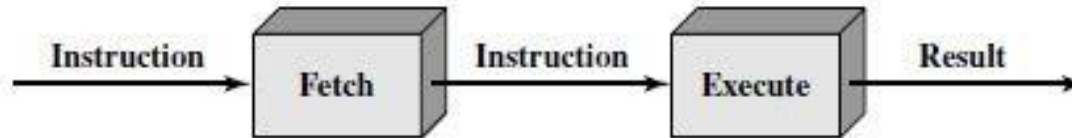


Sequential Execution

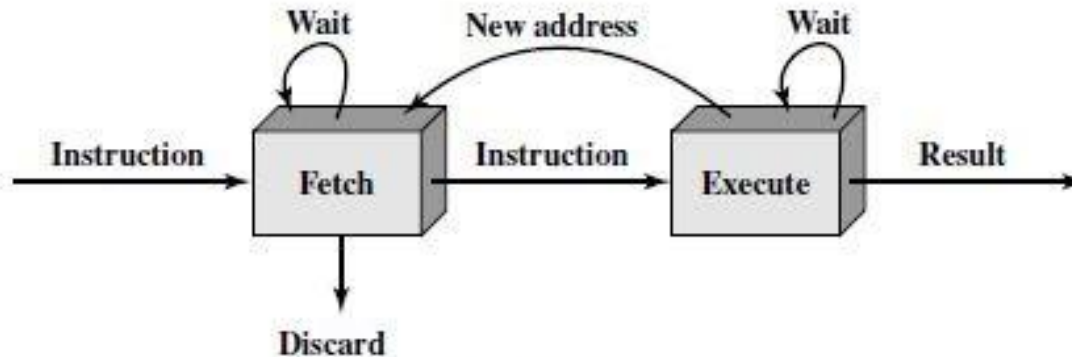


Pipelined Execution

Instruction pipelining-2 stage



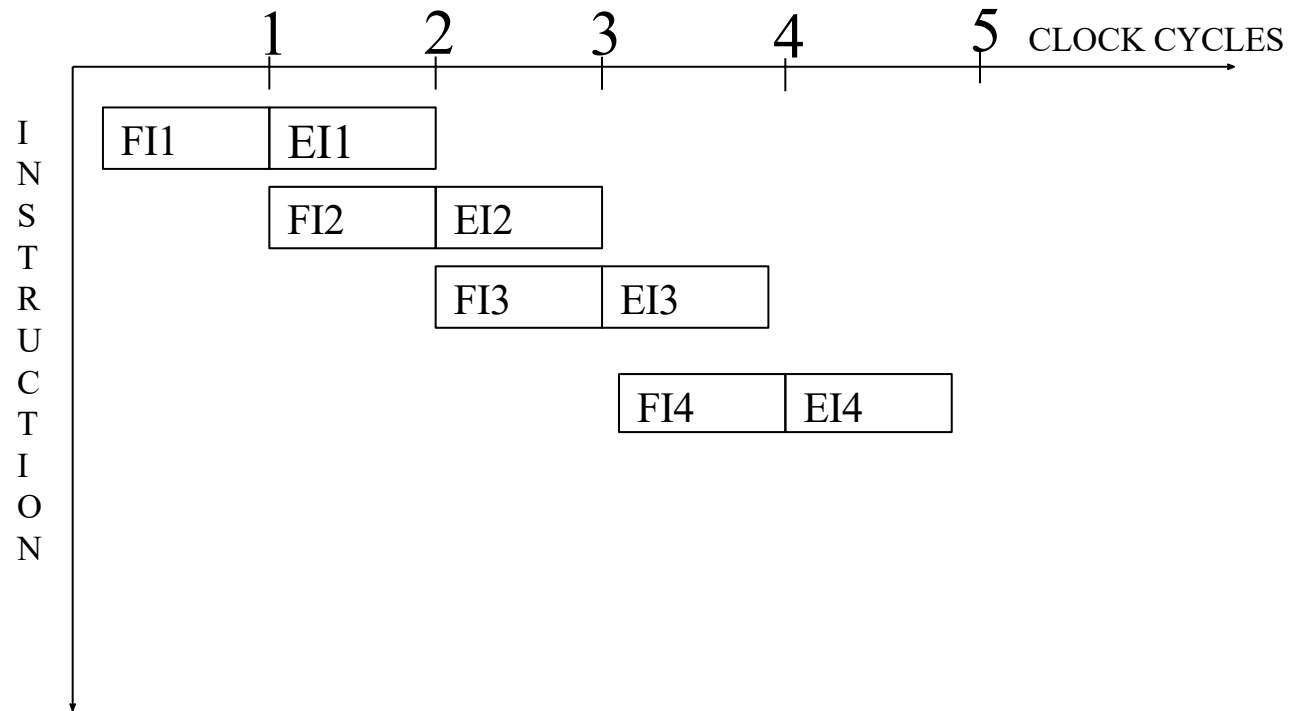
(a) Simplified view



(b) Expanded view

Figure 12.9 Two-Stage Instruction Pipeline

Example---1



- 1.** The execution time will generally be longer than the fetch time. Execution will involve reading and storing operands and the performance of some operation. Thus, the fetch stage may have to wait for some time before it can empty its buffer.
- 2.** A conditional branch instruction makes the address of the next instruction to be fetched unknown.

Thus, the fetch stage must wait until it receives the next instruction address from the execute stage. The execute stage may then have to wait while the next instruction is fetched.

Instruction processing decomposition

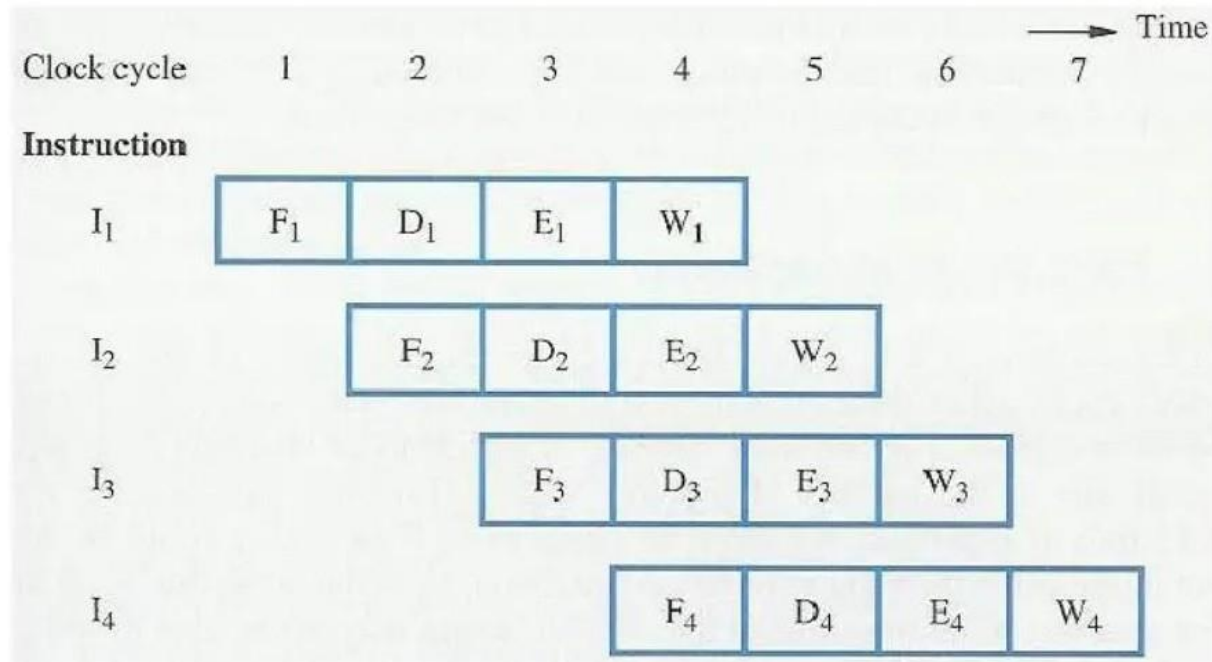
Consider the following decomposition of the instruction processing.

- **Fetch instruction (FI):** Read the next expected instruction into a buffer.
- **Decode instruction (DI):** Determine the opcode and the operand specifiers.
- **Calculate operands (CO):** Calculate the effective address of each source operand. This may involve displacement, register indirect, indirect, or other forms of address calculation.
- **Fetch operands (FO):** Fetch each operand from memory. Operands in registers need not be fetched.
- **Execute instruction (EI):** Perform the indicated operation and store the result, if any, in the specified destination operand location.
- **Write operand (WO):** Store the result in memory.

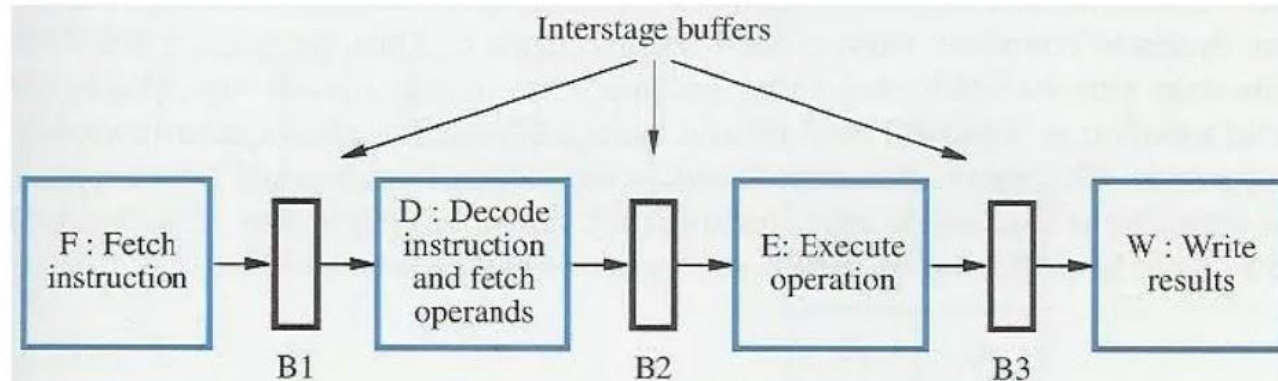
Four State Pipeline

- ◆ Fetch (F)
 - Read the instruction from memory
- ◆ Decode (D)
 - Decode the instruction and fetch the source operand(s)
- ◆ Execute (E)
 - Perform the operation specified by the instruction
- ◆ Write (W)
 - Store the result in the destination location

4-stage pipeline



Hardware Organization



Timing Diagram for Instruction Pipeline

Time →

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO	EI	WO					
Instruction 5					FI	DI	CO	FO	EI	WO				
Instruction 6						FI	DI	CO	FO	EI	WO			
Instruction 7							FI	DI	CO	FO	EI	WO		
Instruction 8								FI	DI	CO	FO	EI	WO	
Instruction 9									FI	DI	CO	FO	EI	WO

Assumptions: Various stages will be of nearly equal duration
each instruction goes through all six stages of the pipeline

The Effect of a Conditional Branch on Instruction Pipeline

Assume that instruction 3 is a conditional branch to instruction 15.

Until the instruction is executed, there is no way of knowing which instruction will come next.

The pipeline simply loads the next instruction in sequence.

At the end of time unit 7 understood that the branch is taken.

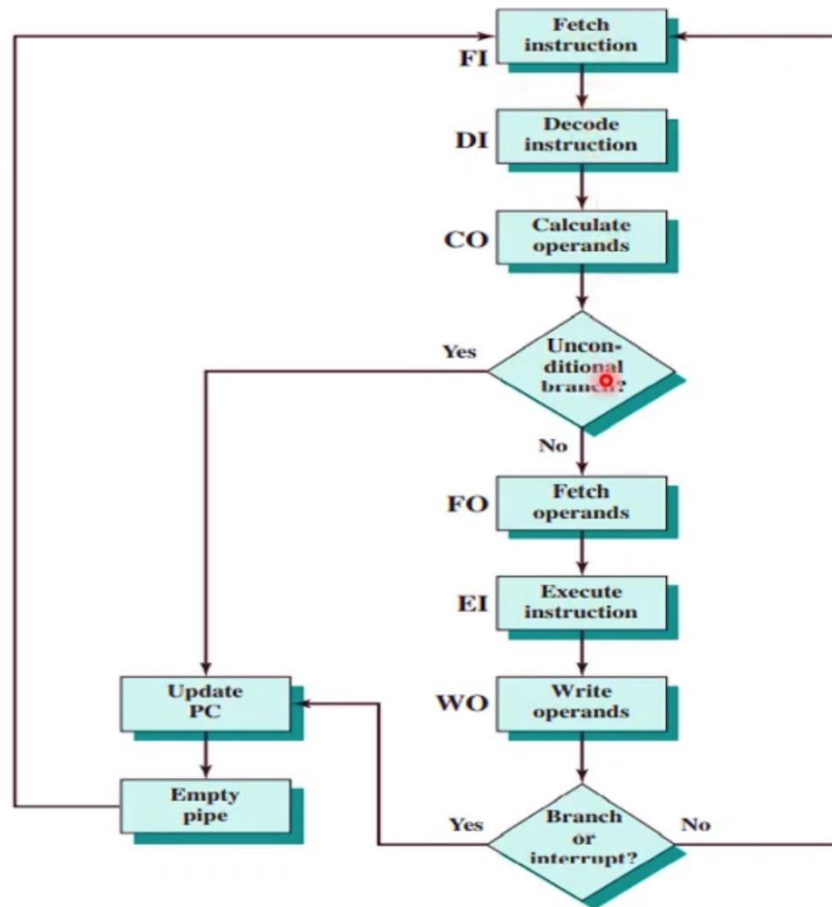
At this point, the pipeline must be cleared of instructions that are not useful.

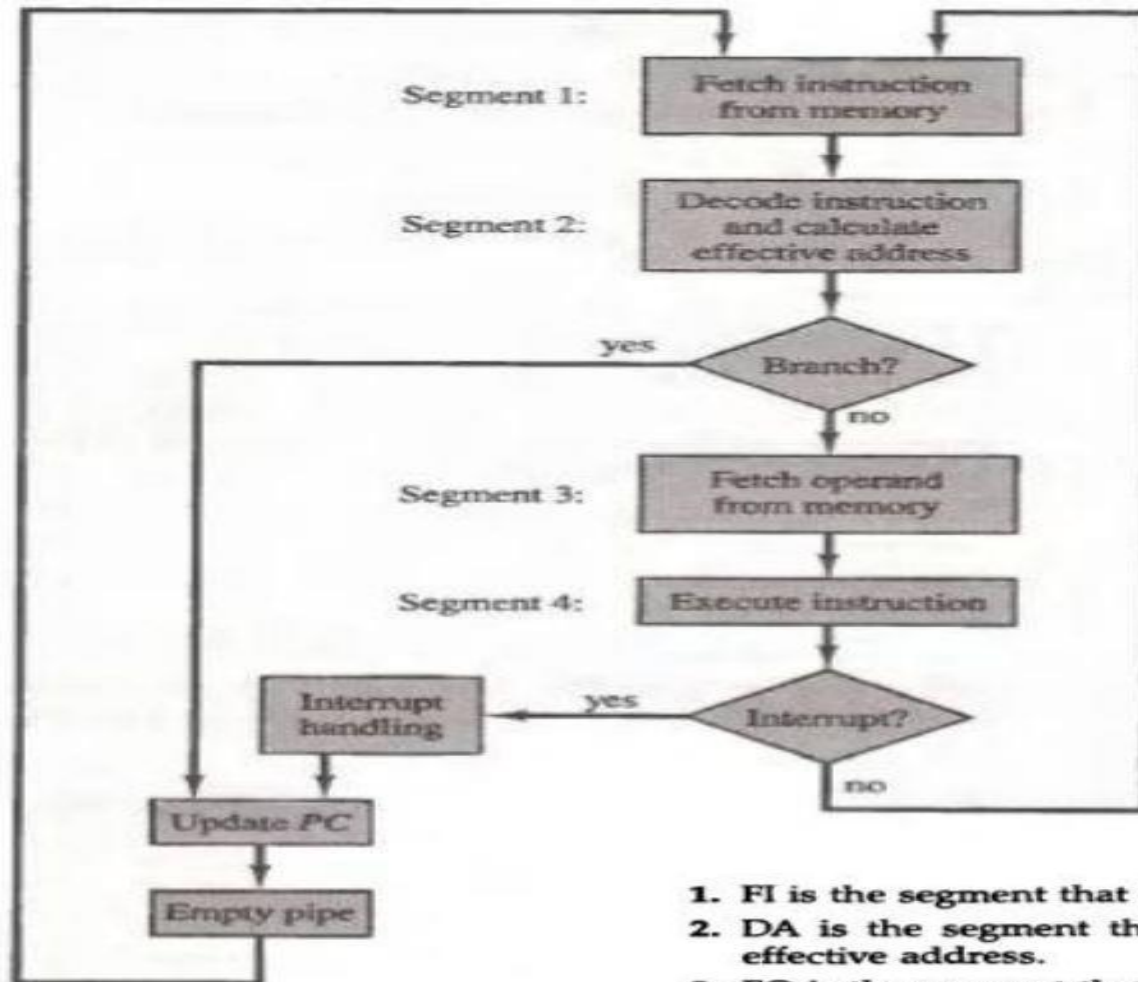
During time unit 8, instruction 15 enters the pipeline.

This is the performance penalty incurred because we could not anticipate the branch

	Time →							← Branch penalty						
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Instruction 1	FI	DI	CO	FO	EI	WO								
Instruction 2		FI	DI	CO	FO	EI	WO							
Instruction 3			FI	DI	CO	FO	EI	WO						
Instruction 4				FI	DI	CO	FO							
Instruction 5					FI	DI	CO							
Instruction 6						FI	DI							
Instruction 7							FI							
Instruction 15								FI	DI	CO	FO	EI	WO	
Instruction 16									FI	DI	CO	FO	EI	WO

Six-Stage CPU Instruction Pipeline





1. FI is the segment that fetches an instruction.
2. DA is the segment that decodes the instruction and calculates effective address.
3. FO is the segment that fetches the operand.
4. EX is the segment that executes the instruction.

Pipeline Hazards

A pipeline hazard occurs when the pipeline, or some portion of the pipeline, must stall because conditions do not permit continued execution.

Such a pipeline stall is also referred to as a pipeline bubble.

□ **Instruction Pipeline hazards**

□ **3 Types of Hazards**

- Resource hazard
 - Access to memory by two segments at the same time
 - Solved by using separate instruction and data memories
- Data hazard
- Control hazard/branch hazard

RESOURCE HAZARDS

Example

- In the ideal case, a new instruction enters the pipeline each clock cycle
- assume that main memory has a **single port** and that all instruction fetches and data reads and writes must be performed one at a time
- In this case, an **operand read** to or write from memory cannot be performed in **parallel with an instruction fetch**
- Therefore, the fetch instruction stage of the pipeline **must idle** for one cycle before beginning the instruction fetch for instruction I3.
- Another example of a resource conflict is a situation in which **multiple instructions** are ready to enter the execute instruction phase and there is a **single ALU**.
- One solutions to such resource hazards is to **increase available resources**, such as having multiple ports into main memory and **multiple ALU** units.

		Clock cycle								
		1	2	3	4	5	6	7	8	9
Instruction	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			FI	DI	FO	EI	WO		
	I4				FI	DI	FO	EI	WO	

(a) Five-stage pipeline, ideal case

		Clock cycle								
		1	2	3	4	5	6	7	8	9
Instruction	I1	FI	DI	FO	EI	WO				
	I2		FI	DI	FO	EI	WO			
	I3			Idle	FI	DI	FO	EI	WO	
	I4					FI	DI	FO	EI	WO

(b) I1 source operand in memory

DATA HAZARDS

- A data hazard occurs when there is a conflict in the access of an operand location.
- In general terms, we can state the hazard in this form: Two instructions in a program are to be executed in sequence and both access a particular memory or register operand.
- If the two instructions are executed in strict sequence, no problem occurs.
- If the instructions are executed in a pipeline, then it is possible for the operand value to be updated producing different result

		Clock cycle									
		1	2	3	4	5	6	7	8	9	10
ADD EAX, EBX		FI	DI	FO	EI	WO					
SUB ECX, EAX			FI	DI	Idle		FO	EI	WO		
13				FI			DI	FO	EI	WO	
14							FI	DI	FO	EI	WO

DATA HAZARDS

- There are three types of data hazards;
- **Read after write (RAW), or true dependency:** An instruction modifies a register or memory location and a succeeding instruction reads the data in that memory or register location. A hazard occurs if the read takes place before the write operation is complete.
- **Write after read (WAR), or antidependency:** An instruction reads a register or memory location and a succeeding instruction writes to the location.
- A hazard occurs if the write operation completes before the read operation takes place.
- **Write after write (WAW), or output dependency:** Two instructions both write to the same location. A hazard occurs if the write operations take place in the reverse order of the intended sequence.

CONTROL HAZARDS

- A **branch hazard**, occurs when the pipeline makes the **wrong decision** on a **branch prediction** and therefore brings instructions into the pipeline that must **subsequently be discarded**.
- A variety of approaches have been taken for dealing with conditional branches:
 - Multiple streams
 - Prefetch branch target
 - Loop buffer
 - Branch prediction
 - Delayed branch



CONTROL HAZARDS HANDLING

MULTIPLE STREAMS

- A simple pipeline suffers a penalty for a branch instruction because it must choose one of two instructions to fetch next and may make the wrong choice.
- It is possible to maintain two pipelines of initial instructions. One pipeline would contain the instructions following the branch and the other pipeline would contain instructions from the target address.
- There are two problems with this approach:
- With multiple pipelines there are contention delays for access to the registers and to memory.
- Additional branch instructions may enter the pipeline (either stream) before the original branch decision is resolved. Each such instruction needs an additional stream.

CONTROL HAZARDS HANDLING

PREFETCH BRANCH TARGET

- When a conditional branch is recognized, the target of the branch is prefetched, in addition to the instruction following the branch.
- This target is then saved until the branch instruction is executed. If the branch is taken, the target has already been prefetched.
- The IBM 360/91 uses this approach.



CONTROL HAZARDS HANDLING

LOOP BUFFER

- A loop buffer is a small, very-high-speed memory maintained by the instruction fetch stage of the pipeline and containing the n most recently fetched instructions, in sequence.
- If a branch is to be taken, the hardware first checks whether the branch target is within the buffer.
- If so, the next instruction is fetched from the buffer.
- CRAY-1 machine uses this approach

CONTROL HAZARDS HANDLING

LOOP BUFFER

■ The loop buffer has three benefits:

- 1) With the use of prefetching, the loop buffer will contain some instructions sequentially ahead of the current instruction fetch address. Thus, instructions fetched in sequence will be available without the usual memory access time.
 - 2) If a branch occurs to a target just a few locations ahead of the address of the branch instruction, the target will already be in the buffer. This is useful for IF–THEN and IF–THEN–ELSE sequences.
 - 3) It is suited to dealing with loops, or iterations; hence the name loop buffer.
- If the loop buffer is large enough to contain all the instructions in a loop, then those instructions need to be fetched from memory only once, for the first iteration. For subsequent iterations, all the needed instructions are already in the buffer.

CONTROL HAZARDS HANDLING

DELAYED BRANCH

It is possible to improve pipeline performance by automatically **rearranging** instructions within a program, so that **branch instructions** occur later than actually desired

BRANCH PREDICTION

Various techniques can be used to predict whether a branch will be taken. Among the more common are the following:

- Predict **never** taken
- Predict **always** taken
- Predict by opcode

Static: do not depend on the execution history

- Taken/**not taken** switch
- Branch **history table**

Dynamic

CONTROL HAZARDS HANDLING

BRANCH PREDICTION

- 1) **Predict never taken:** most popular of all the branch prediction methods.
always assume that the branch will not be taken and continue to fetch instructions in sequence.
- 2) **Predict always taken :** Studies analyzing program behavior have shown that conditional branches are taken more than 50% of the time ,so always prefetching from the branch target address should give better performance than always prefetching from the sequential path.
- 3) **Predict by opcode** The processor assumes that the branch will be taken for certain branch opcodes and not for others. Success rates of greater than 75% is reported for this strategy.

CONTROL HAZARDS HANDLING

BRANCH PREDICTION

Dynamic branch strategies attempt to improve the accuracy of prediction by recording the history of conditional branch instructions in a program.

4) one or more bits can be associated with each conditional branch instruction that reflect the recent history of the instruction. These bits are referred to as a **taken/not taken** switch that directs the processor to make a particular decision the next time the instruction is encountered

5) The **branch history table** is a small cache memory associated with the instruction fetch stage of the pipeline.

Each entry in the table consists of three elements: the address of a branch instruction, some number of history bits that record the state of use of that instruction, and information about the target instruction



Summary

- ◆ Pipelining does not result in individual instructions being executed faster
- ◆ Throughput increases
 - Rate at which instruction execution is completed
- ◆ Important goal in designing processors is to identify all hazards that may cause the pipeline to stall
 - Find ways to minimize their impact

Superscalar operations

Processors that allow several instructions to be executed per clock cycle

Supersalar processor

A superscalar processor is a CPU that implements a form of parallelism called instruction-level parallelism within a single processor.

Superscalar Operation

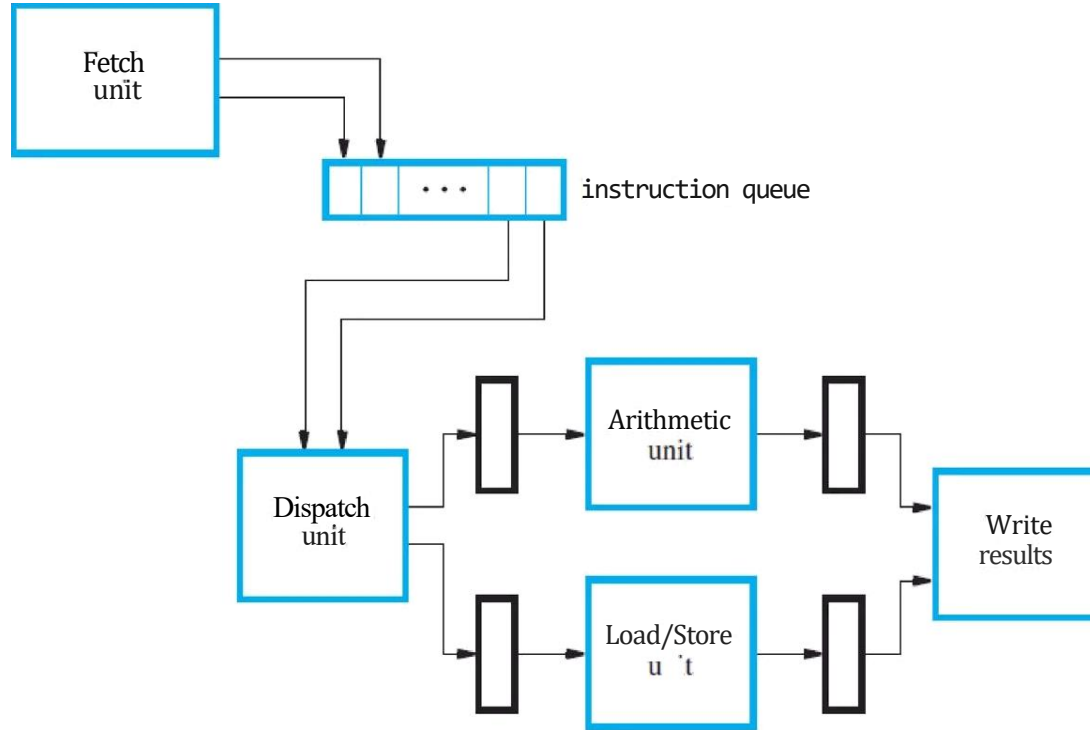


Figure 6. T3 A superscalar processor with two execution units.

- Add R2, R3, #100
- Load R5, 16(R6)
- Subtract R7, R8, R9
- Store R10, 24(R11)

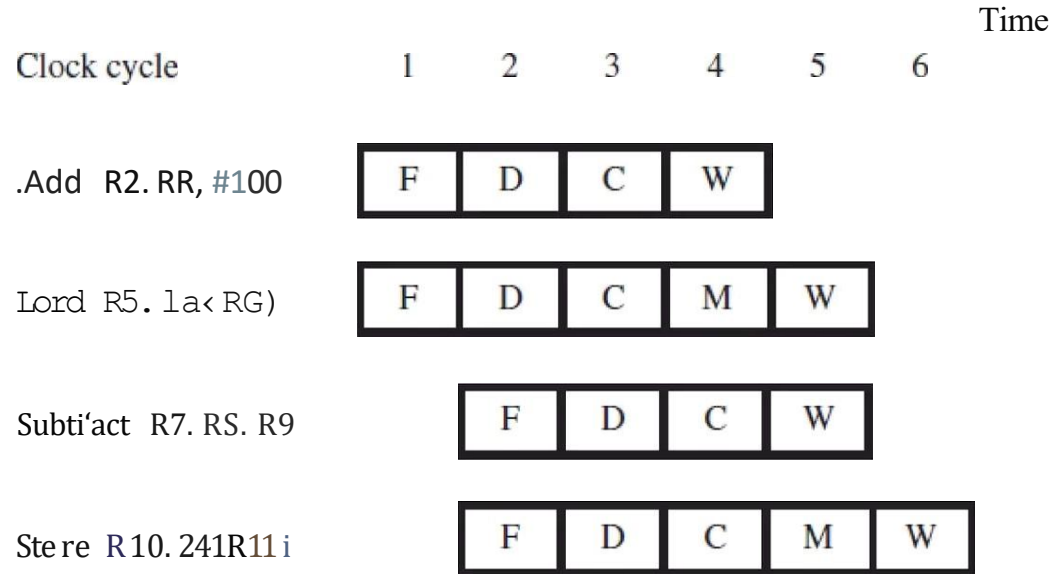


Figure 6.14 An example of instruction flow in the processor of Figure 6.13.

Load store unit
System register unit
Integer unit
Floatingpoint unit

PowerPC 603 pipeline

PowerPC 603 was the
first mainstream
desktop **processor**

